CSP-S 2025 · 第二轮认证 · 提高级

2025 CSP-S 复赛题解

提高组入门版 · 从部分分到满分（修订版）

	得分要求
全国一级	⩾ 131 分
全国二级	⩾ 60 分
全国三级	⩾ 30 分
江苏省一级	⩾ 132 分
江苏省二级	⩾ 60 分
江苏省三级	⩾ 30 分

晋级联赛条件（以江苏省为例，各省政策不同，请以本省通知为准）：

· 高中男生晋级联赛条件：S 组第二轮得分 ⩾ 39 分

· 高中女生晋级联赛条件：S 组第二轮得分 > 0 分

· 初中男生晋级联赛条件：S 组第二轮得分 ⩾ 132 分

· 初中女生晋级联赛条件：S 组第二轮得分 ⩾ 60 分

注意：看分数是不是觉得联赛很好晋级？不是！全省只有 600 ~ 650 个高中生能够有资格参加联赛！

比赛时间：2025 年 11 月 1 日 14:30 ~ 18:30

题目一览：社团招新（club）/ 道路修复（road）/ 谐音替换（replace）/ 员工招聘（employ）

四题时限均为 1.0 秒；内存限制：club / road / employ 为 512 MiB，replace 为 2048 MiB

适合读者：提高组刚入门的同学

目录

写在前面：怎么读这篇题解 3

T1 社团招新（club）—— 贪心 4

数据范围 5

第 1 档：暴力枚举（20 分） 5

第 2 档：特殊性质 A 和 B（再拿 20 分，累计 40 分） 6

第 3 档：满分 —— 先贪心，再"反悔" 6

T2 道路修复（road）—— 最小生成树 + 状态枚举 9

数据范围 9

预备知识：Kruskal 最小生成树 10

第 1 档：k = 0，裸的最小生成树（16 分） 10

第 2 档：特殊性质 A（乡镇免费改造，再拿 28 分，累计 44 分） 11

第 3 档：满分 —— 枚举乡镇子集 + MST 11

T3 谐音替换（replace）—— 字符串哈希 + 字典树 14

数据范围 14

第 1 档：纯暴力（25 分） 15

第 2 档：找到"差异段"，问题立刻变清晰 16

第 3 档：按差异段分组 + 字符串哈希（约 50 分） 16

第 4 档：满分 —— 每组建两棵字典树 17

T4 员工招聘（employ）—— 计数 DP + 容斥 22

数据范围 22

第 1 档：暴力全排列（8 分） 23

第 2 档：把过程抽象出来 24

第 3 档：满分 —— 容斥 + "钦定"思想 24

结语：考场得分策略小结 27

写在前面：怎么读这篇题解

这篇题解是写给刚入门的同学的，每一题都按同样的节奏来讲：

1. **题目原文**：先完整读一遍原题题面（考场如战场，读题是基本功），后面附一句大白话概括帮你核对理解，再给出数据范围表，看清楚每个档次能拿多少分。
2. **分档拿分**：从最容易想到的暴力开始，一档一档往上优化，每改进一步，分数就涨一截。
3. **给出代码**：每一档分析完，都附上可以直接提交的 C++ 代码。

所有代码统一使用万能头 `#include <bits/stdc++.h>` 和 `using namespace std;`，语法尽量朴素，遇到新知识我们会先讲再用。

关于文件读写：CSP 复赛要求用文件输入输出，所以文中每份代码的 `main` 函数开头都有两行 `freopen`（比如 T1 是 `freopen("club.in", "r", stdin);` 和 `freopen("club.out", "w", stdout);`），表示从 `club.in` 读入、向 `club.out` 输出。在自己电脑上练习时，只要在和程序同目录下建一个 `club.in`，把样例输入粘进去，运行后打开生成的 `club.out` 就能看到答案；考到哪个题，就把文件名换成哪个题的英文名即可。

四个题的难度大致是：T1 < T2 < T4 < T3（T3 码量最大，T4 思维最难）。考场上的推荐顺序也是先 T1，再 T2，然后 T4，最后 T3。

对照封面的分数线：拿到任意一道题的较多部分分，就很有希望过三级线；稳稳拿下一道题（100 分）再加一些零散部分分，就摸到一级线了。**部分分非常重要！**

> **各题评测环境（来自官方题面）**：四题每个测试点时限均为 **1.0 秒**；内存限制 club / road / employ 为 **512 MiB**，replace 为 **2048 MiB**。后文分析复杂度时都会对照这个标准。

T1 社团招新（club）—— 贪心

题目原文

【题目描述】小 L 是学校算法协会的成员。在今年的学校社团招新中，小 L 一共招收了 n 个新成员，其中 n 为偶数。现在小 L 希望将他们分到协会不同的部门。

算法协会共设有三个部门，其中第 i (1 ⩽ i ⩽ n) 个新成员对第 j (1 ⩽ j ⩽ 3) 个部门的满意度为 $a_{i,j}$。定义一个分配方案的满意度为所有新成员对分配到的部门的满意度之和，也就是说，若将第 i (1 ⩽ i ⩽ n) 个新成员分配到了第 $d_i \in \{1, 2, 3\}$ 个部门，则该分配方案的满意度为

$$\text{满意度} = a_{1,d_1} + a_{2,d_2} + \ldots + a_{n,d_n}$$

（即：每个人对分配到的那个部门的满意度，全部加起来。）

小 L 不希望某一个部门的新成员数量过多。具体地，他要求在分配方案中，不存在一个部门被分配多于 n/2 个新成员。你需要帮助小 L 求出，满足他要求的分配方案的满意度的最大值。

【输入格式】从文件 club.in 中读入数据。本题包含多组测试数据。输入的第一行包含一个正整数 t，表示测试数据组数。接下来依次输入每组测试数据，对于每组测试数据：第一行包含一个正整数 n；第 i + 1 (1 ⩽ i ⩽ n) 行包含三个非负整数 $a_{i,1}$, $a_{i,2}$, $a_{i,3}$。

【输出格式】输出到文件 club.out 中。对于每组测试数据，输出一行一个非负整数，表示满意度的最大值。

【样例 1 输入】

```
3
4
4 2 1
3 2 4
5 3 4
3 5 1
4
0 1 0
0 1 0
0 2 0
0 2 0
2
10 9 8
4 0 0
```

【样例 1 输出】

```
18
4
13
```

一句话概括：n 个人（n 是偶数）各选一个部门，每个部门不超过 n/2 人，求满意度总和的最大值。

数据范围

对于所有数据：$1 \leqslant t \leqslant 5$；$2 \leqslant n \leqslant 10^5$ 且 n 为偶数；$0 \leqslant a \leqslant 2 \times 10^4$。共 20 个测试点，每个 5 分。时限 1.0 秒，内存 512 MiB。

测试点	n =	特殊性质
1~4	2、4、10、10	无
5~8	30	无
9	200	B
10、11	200	无
12	200	A
13、14	10^5	B
15、16	10^5	C
17 ~ 20	10^5	无

特殊性质 A：所有人对部门 2、3 的满意度都是 0；特殊性质 B：所有人对部门 3 的满意度都是 0；特殊性质 C：满意度随机生成。

第 1 档：暴力枚举（20 分）

n 最小只有 2、4、10。每个人有 3 种选择，一共 3^n 种方案，枚举所有方案，检查人数限制，取最大值即可。$3^{10} \approx 6 \times 10^4$，非常快，可以拿下测试点 1 ~ 4 共 20 分。

```
#include <bits/stdc++.h>
using namespace std;
int t, n;
int a[15][3];
int ans;
void dfs(int i, int c1, int c2, int c3, int sum) {
    if (i == n + 1) {
        if (c1 <= n / 2 && c2 <= n / 2 && c3 <= n / 2)
            ans = max(ans, sum);
        return;
    }
    dfs(i + 1, c1 + 1, c2, c3, sum + a[i][0]);
    dfs(i + 1, c1, c2 + 1, c3, sum + a[i][1]);
    dfs(i + 1, c1, c2, c3 + 1, sum + a[i][2]);
}
int main() {
    freopen("club.in", "r", stdin);
    freopen("club.out", "w", stdout);
    scanf("%d", &t);
    while (t--) {
```

```
        scanf("%d", &n);
        for (int i = 1; i <= n; i++)
            scanf("%d%d%d", &a[i][0], &a[i][1], &a[i][2]);
        ans = 0;
        dfs(1, 0, 0, 0, 0);
        printf("%d\n", ans);
    }
    return 0;
}
```

第 2 档：特殊性质 A 和 B（再拿 20 分，累计 40 分）

特殊性质 B：所有人对部门 3 的满意度都是 0，只剩两个部门。经典贪心：先假设人人都去部门 2，然后挑 n/2 个人"跳槽"到部门 1。第 i 个人跳槽的收益是 a[i][1] - a[i][2]，显然选收益最大的 n/2 个人跳槽最优，按差值排序即可。测试点 9、13、14 共 15 分到手。

特殊性质 A（部门 2、3 满意度都是 0）只是 B 的特例：跳槽收益就是 a[i][1] 本身，同一份代码直接通过，再拿测试点 12 的 5 分。

```
#include <bits/stdc++.h>
using namespace std;
int t, n;
bool cmp(int a, int b) { return a > b; }   // 从大到小
int main() {
    freopen("club.in", "r", stdin);
    freopen("club.out", "w", stdout);
    scanf("%d", &t);
    while (t--) {
        scanf("%d", &n);
        long long sum = 0;
        int dif[100005];
        for (int i = 1; i <= n; i++) {
            int a1, a2, a3;
            scanf("%d%d%d", &a1, &a2, &a3);
            sum += a2;              // 先全部去部门 2
            dif[i] = a1 - a2;       // 跳槽到部门 1 的收益
        }
        sort(dif + 1, dif + n + 1, cmp);
        for (int i = 1; i <= n / 2; i++) sum += dif[i];
        printf("%lld\n", sum);
    }
    return 0;
}
```

这两个性质提示我们一个方向：**先按最理想的方式分配，再修正不合法的地方**。

第 3 档：满分 —— 先贪心，再"反悔"

顺着上面的思路走到底。

第一步：假装没有人数限制。显然每个人都去自己最喜欢的部门最优，设第 i 个人最喜欢部门 d_i，先拿到总满意度 sum。

第二步：检查人数。三个部门里**最多只有一个部门超员**（两个部门都超过 n/2，总人数就超过 n 了，不可能）。如果没有部门超员，sum 就是答案。

第三步：如果部门 d 超员了（设有 cnt 人，cnt > n/2），必须让 cnt - n/2 个人离开部门 d。第 i 个人离开的"损失"是：

$$\text{损失 } loss_i = a[i][d] - \max(a[i][\text{另外两个部门}])。$$

要让总损失最小，当然让损失最小的人先走。把部门 d 里所有人的 loss 从小到大排序，让前 cnt - n/2 小的人离开即可。

为什么这是对的?

- 没有人数限制时，"人人都选最爱"显然最优（每个人独立取最大值）。
- 加了限制后，必须有人离开 d。无论最优方案长什么样，它都等价于"先人人选最爱，再让至少 cnt - n/2 个人离开 d"。每个人离开的损失只和他自己有关、互不影响，所以挑损失最小的人走一定不亏。
- 还有一个细节要检查：离开的人涌向另外两个部门，会不会把那两个部门也挤爆？不会。部门 d 最后恰好剩 n/2 人，所以另外两个部门合起来也只有 n/2 人，每个部门最多 n/2，不会违规。

复杂度：排序一次，O(n log n)，轻松通过 $n \leqslant 10^5$。

```
#include <bits/stdc++.h>
using namespace std;
int t, n;
int a[100005][3];
int main() {
    freopen("club.in", "r", stdin);
    freopen("club.out", "w", stdout);
    scanf("%d", &t);
    while (t--) {
        scanf("%d", &n);
        int cnt[3] = {0, 0, 0};
        int best[100005];
        long long sum = 0;
        for (int i = 1; i <= n; i++) {
            scanf("%d%d%d", &a[i][0], &a[i][1], &a[i][2]);
            int b = 0;
            for (int j = 1; j < 3; j++)
                if (a[i][j] > a[i][b]) b = j;
            best[i] = b;
            cnt[b]++;
            sum += a[i][b];
        }
        // 找超员的部门（最多一个）
        int d = -1;
        for (int j = 0; j < 3; j++)
            if (cnt[j] > n / 2) d = j;
        if (d != -1) {
            // 计算部门 d 里每个人离开的损失
```

```
            long long loss[100005];
            int ln = 0;
            for (int i = 1; i <= n; i++) {
                if (best[i] == d) {
                    long long sec = 0;
                    for (int j = 0; j < 3; j++)
                        if (j != d && a[i][j] > sec) sec = a[i][j];
                    loss[ln++] = a[i][d] - sec;
                }
            }
            sort(loss, loss + ln);
            for (int i = 0; i < cnt[d] - n / 2; i++)
                sum -= loss[i];
        }
        printf("%lld\n", sum);
    }
    return 0;
}
```

用样例第一组验证：4 个人分别选部门 1、3、1、2，各部门人数 2、1、1，没人超员，答案 4 + 4 + 5 + 5 = 18，和样例一致。

T2 道路修复（road）—— 最小生成树 + 状态枚举

题目原文

【题目描述】C 国的交通系统由 n 座城市与 m 条连接两座城市的双向道路构成，第 i ($1 \leqslant i \leqslant m$) 条道路连接城市 u_i 和 v_i。任意两座城市都能通过若干条道路相互到达。

然而，近期由于一场大地震，所有 m 条道路都被破坏了，修复第 i 条道路的费用为 w_i。与此同时，C 国还有 k 个准备进行城市化改造的乡镇。对于第 j ($1 \leqslant j \leqslant k$) 个乡镇，C 国对其进行城市化改造的费用为 c_j。在城市化改造完第 j 个乡镇后，可以在这个乡镇与原来的 n 座城市间建造若干条道路，其中在它与第 i 座城市间建造一条道路的费用为 $a_{j,i}$。C 国可以在这 k 个乡镇中选择任意多个进行城市化改造，也可以不选择任何乡镇。

为尽快恢复城市间的交通，C 国政府希望以最低的费用将原有的 n 座城市两两连通。你需要帮助他们求出最小费用。

【输入格式】从文件 road.in 中读入数据。第一行包含三个非负整数 n, m, k。第 i + 1 ($1 \leqslant i \leqslant m$) 行包含三个非负整数 u_i, v_i, w_i。第 j + m + 1 ($1 \leqslant j \leqslant k$) 行包含 n + 1 个非负整数 c_j, $a_{j,1}$, $a_{j,2}$, …, $a_{j,n}$。

【输出格式】输出到文件 road.out 中，一行一个非负整数。

【样例 1 输入】

```
4 4 2
1 4 6
2 3 7
4 2 5
4 3 4
1 1 8 2 4
100 1 3 2 4
```

【样例 1 输出】

```
13
```

一句话概括：n 座城市、m 条可修复道路，还有 $k \leqslant 10$ 个乡镇可花钱改造并修路，求让所有城市两两连通的最小花费。$k \leqslant 10$ 是全场最大的提示：$2^{10} = 1024$，可以枚举每个乡镇"改造还是不改造"！

数据范围

对于所有数据：$1 \leqslant n \leqslant 10^4$，$1 \leqslant m \leqslant 10^6$，$0 \leqslant k \leqslant 10$；边权和费用不超过 10^9；原有城市之间保证连通。共 25 个测试点，每个 4 分。时限 1.0 秒，内存 512 MiB。

测试点	n ⩽	m ⩽	k ⩽	特殊性质

1~4	10^4	10^6	0	无
5、6	10^4	10^5	5	A
7、8	10^4	10^5	5	无
9、10	10^3	10^5	5	A
11、12	10^3	10^5	5	无
13、14	10^4	10^6	10	A
15、16	10^4	10^6	10	无
17、18	10^4	10^5	5	A
19、20	10^4	10^5	5	无
21 ~ 25	10^4	10^6	10	无

特殊性质 A：每个乡镇改造费 $c_j = 0$，且都至少有一条 $a_{j,i} = 0$ 的边。

预备知识：Kruskal 最小生成树

本题的地基是最小生成树（MST）。Kruskal 算法三步走：

1. 把所有边按边权从小到大排序；
2. 从小到大依次看每条边，如果它连接的两个点还不连通，就选它（用并查集判断）；
3. 选了 n - 1 条边后，得到的就是连接所有点、总边权最小的方案。

不会的同学建议先把并查集和 Kruskal 学明白，这是提高组必备基本功。

第 1 档：k = 0，裸的最小生成树（16 分）

测试点 1 ~ 4 没有乡镇，就是裸的最小生成树，Kruskal 模板直接过，16 分到手。

> **注意**：下面的代码面向 k = 0 的测试点 1~4。为了让它在任何输入下都能正确运行（也方便你拿题目自带的样例自测），代码会把乡镇数据**读进来再丢掉**——k = 0 时这个循环什么都不做，不影响得分。

```
#include <bits/stdc++.h>
using namespace std;
struct Edge { int u, v; long long w; };
Edge es[1000005];
int fa[10005];
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
bool cmp(Edge a, Edge b) { return a.w < b.w; }   // 按边权从小到大
int main() {
    freopen("road.in", "r", stdin);
```

```
    freopen("road.out", "w", stdout);
    int n, m, k;
    scanf("%d%d%d", &n, &m, &k);
    for (int i = 0; i < m; i++)
        scanf("%d%d%lld", &es[i].u, &es[i].v, &es[i].w);
    // 把乡镇数据读掉（本题档答案与乡镇无关，读进来丢弃即可）
    for (int j = 0; j < k; j++)
        for (int i = 0; i <= n; i++) {
            long long x; scanf("%lld", &x);
        }
    sort(es, es + m, cmp);
    for (int i = 1; i <= n; i++) fa[i] = i;
    long long ans = 0;
    for (int i = 0; i < m; i++) {
        int x = find(es[i].u), y = find(es[i].v);
        if (x != y) { fa[x] = y; ans += es[i].w; }
    }
    printf("%lld\n", ans);
    return 0;
}
```

第 2 档：特殊性质 A（乡镇免费改造，再拿 28 分，累计 44 分）

性质 A 保证 $c_j = 0$ 且每个乡镇都至少有一条 $a_{j,i} = 0$ 的边。改造不要钱，而那条 0 费用的边保证把乡镇"挂"进网络也不花一分钱，所以**把所有乡镇全部改造一遍一定不亏**（0 边先免费连通乡镇，其余乡镇边只会让连通城市更便宜）：把 k 个乡镇也当成普通节点，连同它们到城市的边一起丢进 Kruskal 即可。

性质 A 的测试点是 5、6、9、10、13、14、17、18，共 7 个点 × 4 分 = **28 分**，与第 1 档累计 44 分。代码与第 1 档几乎相同：把乡镇编号为 n + 1 ~ n + k，把所有乡镇边加入边表即可，此处不再重复贴出。

这提示我们满分思路的雏形：**只要确定了"改造哪些乡镇"，剩下的就是一次最小生成树**。

第 3 档：满分 —— 枚举乡镇子集 + MST

核心思路：先"开天眼"，再枚举。我们把问题反过来想，一步一步推。

先假设我们"开了天眼"，知道最优方案用了哪些乡镇，把这个集合记作 T。那么这个最优方案一定长这样：花 c(T) 改造 T 里的乡镇，然后在"城市 + T 里的乡镇"这些点之间，用一种总费用最低的方式把大家连通。

为什么修路部分一定是 MST？因为最优方案修好的那些路（包括连乡镇的新路），确实把"城市 ∪ T"全部连通了；而在同一个点集上，费用最低的连通方案就是最小生成树——如果它修路的部分不是 MST，我们换成 MST 只会更便宜，那它就不配叫"最优方案"了。所以：

$$\text{知道 T} \Longrightarrow \text{最优费用} = c(T) + \text{MST}(\text{城市} \cup T)。$$

可惜我们没有天眼，不知道 T 是哪个集合。没关系——$k \leqslant 10$，一共才 $2^k \leqslant 1024$ 个集合，全部试一遍，取最小值就行了。试到 S = T 的时候，算出来的恰好就是最优解的费用，所以最小值一定就是答案。

最后打消两个担心：

- "试到别的集合 S，会不会算出比最优解还小的假答案？"不会。c(S) + MST(城市 ∪ S) 本身就是一个真实可行的修法（改造 S，再按 MST 修路），真实方案的费用怎么可能比最优解还低呢？
- "MST 会把 S 里所有乡镇都连进树里，万一连了个没用的'叶子'乡镇，白花钱怎么办？"那只会让这个候选值偏大，不影响全局——我们要取的是最小值，真正有用的集合 T 给出的候选值才是最小的那个。

剩下的问题是复杂度：m 高达 10^6，如果每个子集都把 10^6 条边重新跑一遍 Kruskal，1024×10^6 在 1 秒时限内直接超时。注意到一个经典结论：

> **图 G 的 MST 加上任意新边集 E′，新图的 MST 一定只用"原 MST 的边 + E′ 里的边"。**
> （原 MST 里被替换掉的边，用交换论证可以证明新图 MST 不需要它们。）

所以我们先对原图跑一次 Kruskal，把 10^6 条边浓缩成 n - 1 条（$n \leqslant 10^4$）。之后每个子集要处理的边只有：n - 1 条原图边 + $|S| \times n$ 条乡镇边 $\approx 6 \times 10^4$ 条。

进一步优化：把每个乡镇到 n 座城市的边预先各排好一次序（只排 k 次）。原 MST 边也是有序的。这样每个子集只需把 $|S| + 1$ 个有序边表做多路归并（用小根堆），不用重新排序，每合并出一条边就做一次并查集操作。总复杂度约为 $O(2^k \cdot k \cdot n \cdot \log k)$，约 10^8 次简单操作、常数很小，官方评测机（1.0 秒时限）下可以通过——官方标程也是同样的复杂度。

```
#include <bits/stdc++.h>
using namespace std;
struct Edge { int u, v; long long w; };
int n, m, k;
int fa[20010];
int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
Edge es[1000005];              // 原图所有边
Edge mstE[10005];              // 原图 MST 的边（已有序）
int mstN = 0;
pair<long long, int> te[10][10005];  // te[j][i]：乡镇 j 到城市 i 的边 (费用, 城市)
long long cst[10];             // 改造费用
bool cmp(Edge a, Edge b) { return a.w < b.w; }   // 按边权从小到大

int main() {
    freopen("road.in", "r", stdin);
    freopen("road.out", "w", stdout);
    scanf("%d%d%d", &n, &m, &k);
    for (int i = 0; i < m; i++)
        scanf("%d%d%lld", &es[i].u, &es[i].v, &es[i].w);
    for (int j = 0; j < k; j++) {
        scanf("%lld", &cst[j]);
        for (int i = 1; i <= n; i++) {
            long long x; scanf("%lld", &x);
            te[j][i] = make_pair(x, i);
        }
        sort(te[j] + 1, te[j] + n + 1);   // 每个乡镇的边预排序
    }
```

```
    // 原图浓缩成 MST
    sort(es, es + m, cmp);
    for (int i = 1; i <= n; i++) fa[i] = i;
    for (int i = 0; i < m; i++) {
        int x = find(es[i].u), y = find(es[i].v);
        if (x != y) { fa[x] = y; mstE[mstN++] = es[i]; }
    }
    long long ans = LLONG_MAX;
    // 枚举乡镇子集
    for (int S = 0; S < (1 << k); S++) {
        int towns[10], tn = 0;
        long long cost = 0;
        for (int j = 0; j < k; j++)
            if (S >> j & 1) { towns[tn++] = j; cost += cst[j]; }
        int tot = n + tn;             // 总点数: 城市 + 选中乡镇
        for (int i = 1; i <= tot; i++) fa[i] = i;
        // 多路归并: 源 0 = 原 MST 边; 源 s+1 = 乡镇 towns[s] 的边
        // 大根堆里存 (-边权, 来源), 就相当于小根堆
        priority_queue<pair<long long, int> > pq;
        int ptr[11];
        for (int s = 0; s <= tn; s++) ptr[s] = (s == 0 ? 0 : 1);
        if (mstN > 0) pq.push(make_pair(-mstE[0].w, 0));
        for (int s = 0; s < tn; s++)
            pq.push(make_pair(-te[towns[s]][1].first, s + 1));
        int added = 0, need = tot - 1;
        while (added < need && !pq.empty()) {
            pair<long long, int> top = pq.top(); pq.pop();
            long long w = -top.first;
            int src = top.second, u, v;
            if (src == 0) {
                u = mstE[ptr[0]].u; v = mstE[ptr[0]].v;
                ptr[0]++;
                if (ptr[0] < mstN) pq.push(make_pair(-mstE[ptr[0]].w, 0));
            } else {
                int j = towns[src - 1];
                u = n + src; v = te[j][ptr[src]].second;
                ptr[src]++;
                if (ptr[src] <= n)
                    pq.push(make_pair(-te[j][ptr[src]].first, src));
            }
            int x = find(u), y = find(v);
            if (x != y) { fa[x] = y; cost += w; added++; }
        }
        if (cost < ans) ans = cost;
    }
    printf("%lld\n", ans);
    return 0;
}
```

样例验证：枚举到只改造乡镇 1 时，改造费 1，MST 选边 乡镇1–城市1（1）、乡镇1–城市3（2）、道路 4–3（4）、道路 4–2（5），合计 1 + 1 + 2 + 4 + 5 = 13，与样例输出一致。

T3 谐音替换（replace）—— 字符串哈希 + 字典树

题目原文

【题目描述】给定 n 个字符串二元组，第 i ($1 \leqslant i \leqslant n$) 个为 ($s_{i,1}$, $s_{i,2}$)，满足 $|s_{i,1}| = |s_{i,2}|$。

对于字符串 s，定义 s 的替换如下：对于 s 的某个子串 y，若存在 $1 \leqslant i \leqslant n$ 满足 $y = s_{i,1}$，则将 y 替换为 $y' = s_{i,2}$。具体地，设 $s = x + y + z$（x、z 可以为空），则替换得到 $s' = x + y' + z$。

小 W 提出了 q 个问题，第 j 个问题给定两个**不同**的字符串 $t_{j,1}$, $t_{j,2}$，她想知道有多少种 $t_{j,1}$ 的替换能够得到 $t_{j,2}$。两种替换不同当且仅当子串 y 的位置不同或用于替换的二元组不同（即 x、z 不同或 i 不同）。

【输入格式】从文件 replace.in 中读入数据。第一行包含 n, q；第 i + 1 行包含 $s_{i,1}$, $s_{i,2}$；第 j + n + 1 行包含 $t_{j,1}$, $t_{j,2}$。

【输出格式】输出到文件 replace.out 中，输出 q 行。

【样例 1 输入】

```
4 2
xabcx xadex
ab cd
bc de
aa bb
xabcx xadex
aaaa bbbb
```

【样例 1 输出】

```
2
0
```

【样例 1 解释】对于第一个询问，共有 2 种替换：

1. 令 x, z 均为空串，y = xabcx，i = 1，则 y' = xadex；
2. 令 x = xa，y = bc，z = x，i = 3，则 y' = de。

一句话概括：每条规则可以把 s 中等于 $s_{i,1}$ 的子串换成 $s_{i,2}$；每个询问问 t1 有多少种"一次替换"能变成 t2（替换位置不同或规则不同就算不同）。

数据范围

设 L1 为所有规则串总长，L2 为所有询问串总长。对于所有数据：$1 \leqslant n, q \leqslant 2 \times 10^5$；$2 \leqslant L1, L2 \leqslant 5 \times 10^6$；$|s_{i1}| = |s_{i2}|$，t1 ≠ t2。共 20 个测试点，每个 5 分。时限 1.0 秒，内存 2048 MiB。

测试点	n, q ⩽	L1, L2 ⩽	特殊性质

1、2	10^2	200	无
3~5	10^3	2000	无
6	10^3	10^6	A、B
7、8	10^4	10^6	A
9、10	2×10^5	10^6	B
11、12	2×10^5	2×10^6	无
13、14	2×10^5	5×10^6	A
15、16	2×10^5	5×10^6	B
17 ~ 20	2×10^5	5×10^6	无

特殊性质 A：$q = 1$；特殊性质 B：所有字符串只含 a、b 且 b 恰好出现一次。

第 1 档：纯暴力（25 分）

测试点 1、2 中 $n, q \leqslant 100$，总长度 $\leqslant 200$；测试点 3 ~ 5 中 $n, q \leqslant 1000$，所有字符串总长也不超过 2000，暴力同样够快。直接四重循环：枚举询问、枚举规则、枚举替换位置、逐字符比对。

一个关键观察可以先帮我们剪枝：**替换不改变长度**（因为 $|s_{i1}| = |s_{i2}|$），所以 $|t1| \neq |t2|$ 时答案一定是 0。

```
#include <bits/stdc++.h>
using namespace std;
int n, q;
string s1[1005], s2[1005];
int main() {
    freopen("replace.in", "r", stdin);
    freopen("replace.out", "w", stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> q;
    for (int i = 1; i <= n; i++) cin >> s1[i] >> s2[i];
    while (q--) {
        string t1, t2;
        cin >> t1 >> t2;
        int ans = 0;
        if (t1.size() == t2.size()) {
            for (int i = 1; i <= n; i++) {              // 枚举规则
                int len = s1[i].size();
                for (int p = 0; p + len <= (int)t1.size(); p++) {  // 枚举位置
                    if (t1.substr(p, len) != s1[i]) continue;
                    string r = t1.substr(0, p) + s2[i] + t1.substr(p + len);
                    if (r == t2) ans++;
                }
            }
        }
```

```
        printf("%d\n", ans);
    }
    return 0;
}
```

第 2 档：找到"差异段"，问题立刻变清晰

这是本题最重要的一步观察，也是通往满分的桥梁。比较 t1 和 t2，设它们第一处不同的位置是 L，最后一处不同的位置是 R。一次替换要能把 t1 变成 t2，被替换的子串必须**完全盖住 [L, R]** 这段（否则那段没法变）；而 [L, R] 之外的部分，替换前后必须一模一样。

于是可以把 t1 拆成三段：

$$t1 = 前缀\ TA + 差异段\ [L..R] + 后缀\ TC。$$

同样，把每条规则 (s1, s2) 也做一样的拆分：找到 s1 和 s2 的第一处不同 a、最后一处不同 b，拆成：

$$s1 = 规则前缀\ pre + 差异段\ core1 + 规则后缀\ suf$$
$$s2 = 规则前缀\ pre + 差异段\ core2 + 规则后缀\ suf$$

（注意：差异段之外 s1 和 s2 完全相同，所以 pre、suf 是共享的。）

这条规则能对这次询问生效，当且仅当三条同时成立：

1. **差异段完全对应**：core1 = t1[L..R] 且 core2 = t2[L..R]；
2. 规则前缀 pre 是 TA 的**后缀**（位置要接得上）；
3. 规则后缀 suf 是 TC 的**前缀**。

而且一旦三条都满足，替换位置被唯一确定（pre 多长，位置就定死了），所以每条满足条件的规则恰好贡献 1 种方案。问题变成：**数一数有多少条规则同时满足这三条**。

第 3 档：按差异段分组 + 字符串哈希（约 50 分）

条件 1 只跟"差异段从什么变成什么"有关。我们用**字符串哈希**把每对 (core1, core2) 压成一个数，把规则按这个数分组；询问也算出自己的 (core1, core2)，只和同组的规则比。

> **字符串哈希小科普**：把字符串看成 base 进制数，$h = s_0 \cdot base^{len-1} + s_1 \cdot base^{len-2} + \cdots$，用 unsigned long long 自然溢出当取模。两个哈希值相等就（几乎）可以认为字符串相等。预处理前缀哈希后，任意子串的哈希都能 O(1) 算出来。

分组之后，条件 2、3 还是要逐条规则检查。借助 pre 和 suf 的哈希，每次检查 O(1)，即每个询问 O(n)。它能拿下哪些测试点？

- q = 1 的特殊性质 A（测试点 7、8、13、14）：每个询问 O(n)，完全够用；
- 测试点 6（$n, q \leqslant 10^3$）：$O(nq) \leqslant 10^6$，轻松通过。

注意测试点 9、10 虽然有特殊性质 B，但 $n, q \leqslant 2 \times 10^5$，$O(nq) \approx 4 \times 10^{10}$ 必然超时，**拿不到**。配合前面的暴力（测试点 1~5），此时累计能拿下 25 + 5 + 10 + 10 = **约 50 分**。

但一般数据中 n, q 都是 2×10^5，O(nq) 不行。我们需要让同组内"pre 是 TA 后缀且 suf 是 TC 前缀"的规则计数也变得飞快。

第 4 档：满分 —— 每组建两棵字典树

还是围绕条件 2、3 做文章。先看一个化简小技巧：

> "pre 是 TA 的后缀"这件事，把两个串都反过来写，就变成了"reverse(pre) 是 reverse(TA) 的前缀"。

这样一来，条件 2、3 就统一成了两个"前缀匹配"：

1. reverse(pre) 是 U = reverse(TA) 的前缀；
2. suf 是 TC 的前缀。

> **字典树（Trie）小科普**：Trie 是一棵专门存字符串的树，从根出发每条边是一个字符，从根到某个节点的路径拼起来就是一个字符串。它最大的本事是：一个串 s 是另一个串 t 的前缀，当且仅当把 t 放到 Trie 上走时，s 的结尾节点正好落在经过的路径上。判断"前缀匹配"就是 Trie 的拿手好戏。

第一棵 Trie：管 pre。同一组的规则和询问放在一起离线处理。把组内所有规则的 reverse(pre) 插进 Trie1。对某个询问，把 U = reverse(TA) 放上去走，走到节点 u（走断了就停在能走到的最深节点）——那么"pre 是 TA 后缀"的规则，恰好就是结尾挂在**根到 u 这条路径上**的那些。

难的地方来了：路径上可能有很多条规则，其中还要筛出"suf 是 TC 前缀"的。难道每条都检查一遍？

关键技巧：DFS 这棵树，边走边维护第二棵 Trie。我们对 Trie1 做一次 DFS（就是普通的遍历整棵树），并维护一个"当前候选池"——也用一棵字典树 Trie2 来存：

- 进入一个节点时，把挂在这个节点上的规则的 suf 插入 Trie2（结尾节点计数 +1）；
- 离开一个节点时，把刚才插入的 suf 计数 −1（撤销）。

想一想 DFS 的性质：DFS 进行到节点 u 时，"已经进入但还没离开"的节点，恰好是根到 u 的这条路径。所以此刻 Trie2 里装着的，正好就是"pre 是 TA 后缀"的全部候选规则的 suf！

于是把每个询问也挂到 Trie1 的对应节点上（U 走到的节点），DFS 经过时顺手回答它：把 TC 放到 Trie2 上走，把沿途节点的计数全部加起来（走断就停），就是答案。一模一样的规则会重复计数，刚好符合题意。

复杂度：每条规则的 suf 被插入、撤销各一次，$O(|suf|)$；建树和 DFS 都是 O(节点数)，节点总数不超过所有规则串总长；每个询问回答一次 $O(|TC|)$。加起来是 O(L1 + L2)，完全线性。内存方面两棵 Trie 的节点总数不超过 $\Sigma|s1| \leqslant 2.5 \times 10^6$（$L1 \leqslant 5 \times 10^6$ 是 s1、s2 的总长，s1 只占一半），每个节点 26 个指针，总共约 500 MB，本题内存限制 2048 MiB，放心开。

> **为什么节点数必须卡紧？** 实际运行的大头开销是建树时每个节点对 26 个指针的清零（约 $2.5 \times 10^6 \times 104\ \text{B} \approx 2.6 \times 10^8$ 字节写入）。正因为节点上界取的是 $\Sigma|s1| \leqslant 2.5 \times 10^6$ 而不是 L1，这份开销才在 1 秒时限内可以接受。

几个细节：

- $s_{i1} = s_{i2}$ 的规则没有差异段，换上等于没换，直接丢掉。
- 询问里 $|t1| \neq |t2|$ 直接输出 0。
- 哈希值相同就视为同组，理论上有极小碰撞概率，比赛中可以放心使用（担心的话可以写双哈希）。

```
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull;
const ull base = 131;
int n, q;

// ---------- 字符串哈希 ----------
ull hstr(string s) {
    ull h = 0;
    for (int i = 0; i < (int)s.size(); i++) h = h * base + (s[i] - 'a' + 1);
    return h;
}

// ---------- 两棵字典树 ----------
// Trie1 存 reverse(pre)，Trie2 存 suf（DFS 过程中动态增删）
const int MAXV = 2500005;
int ch1[MAXV][26], n1;
int ch2[MAXV][26], cnt2[MAXV], n2;
int rhead[MAXV], qhead[MAXV];
int newNode1() { memset(ch1[n1], 0, sizeof(ch1[n1])); rhead[n1] = qhead[n1] = -1; return
n1++; }
int newNode2() { memset(ch2[n2], 0, sizeof(ch2[n2])); cnt2[n2] = 0; return n2++; }

// ---------- 数据 ----------
map<ull, int> g2id;
vector<string> rpre, rsuf;        // 规则：reverse(pre), suf
vector<vector<int> > grules;     // 每组包含的规则编号
vector<string> qU, qTC;           // 询问：reverse(TA), TC
vector<long long> ans;
vector<vector<int> > gqs;        // 每组包含的询问编号
vector<int> rnxt, qnxt;           // 挂链表（规则和询问各一条）

// enter(u)：把挂在 u 上的规则的 suf 插入 Trie2，并回答挂在 u 上的询问
void enter(int u) {
    int id, v, p, x;
    long long s;
    for (id = rhead[u]; id != -1; id = rnxt[id]) {
        v = 0;
        for (p = 0; p < (int)rsuf[id].size(); p++) {
            x = rsuf[id][p] - 'a';
            if (!ch2[v][x]) ch2[v][x] = newNode2();
            v = ch2[v][x];
```

```
        }
        cnt2[v]++;
    }
    for (id = qhead[u]; id != -1; id = qnxt[id]) {
        v = 0; s = cnt2[0];        // suf 为空的规则在根节点计数
        for (p = 0; p < (int)qTC[id].size(); p++) {
            x = qTC[id][p] - 'a';
            if (!ch2[v][x]) break;
            v = ch2[v][x];
            s += cnt2[v];
        }
        ans[id] = s;
    }
}
// quit(u)：撤销挂在 u 上的规则的 suf 计数
void quit(int u) {
    int id, v, p;
    for (id = rhead[u]; id != -1; id = rnxt[id]) {
        v = 0;
        for (p = 0; p < (int)rsuf[id].size(); p++)
            v = ch2[v][rsuf[id][p] - 'a'];
        cnt2[v]--;
    }
}

int main() {
    freopen("replace.in", "r", stdin);
    freopen("replace.out", "w", stdout);
    scanf("%d%d", &n, &q);
    // 读入规则，按差异段哈希分组
    for (int i = 0; i < n; i++) {
        static char buf1[5000005], buf2[5000005];
        scanf("%s%s", buf1, buf2);
        string s1 = buf1, s2 = buf2;
        int len = s1.size(), a = -1, b = -1;
        for (int p = 0; p < len; p++)
            if (s1[p] != s2[p]) { if (a == -1) a = p; b = p; }
        if (a == -1) continue;    // s1 == s2，无用规则
        string pre = s1.substr(0, a);
        string suf = s1.substr(b + 1);
        ull key = hstr(s1.substr(a, b - a + 1) + "{" + s2.substr(a, b - a + 1));
        if (!g2id.count(key)) {
            g2id[key] = grules.size();
            grules.push_back(vector<int>());
            gqs.push_back(vector<int>());
        }
        reverse(pre.begin(), pre.end());   // 后缀变前缀
        int g = g2id[key];
        grules[g].push_back(rpre.size());
        rpre.push_back(pre);
        rsuf.push_back(suf);
    }
    // 读入询问，同样按差异段分组（离线处理）
    ans.assign(q, 0);
    for (int i = 0; i < q; i++) {
```

```
        static char buf1[5000005], buf2[5000005];
        scanf("%s%s", buf1, buf2);
        string t1 = buf1, t2 = buf2;
        if (t1.size() != t2.size()) { qU.push_back(""); qTC.push_back(""); continue; } //
占位对齐下标
        int len = t1.size(), L = -1, R = -1;
        for (int p = 0; p < len; p++)
            if (t1[p] != t2[p]) { if (L == -1) L = p; R = p; }
        ull key = hstr(t1.substr(L, R - L + 1) + "{" + t2.substr(L, R - L + 1));
        if (!g2id.count(key)) { qU.push_back(""); qTC.push_back(""); continue; }  // 无此
组，答案为 0
        string TA = t1.substr(0, L);
        reverse(TA.begin(), TA.end());
        qU.push_back(TA);
        qTC.push_back(t1.substr(R + 1));
        gqs[g2id[key]].push_back(i);
    }
    rnxt.assign(rpre.size(), -1);
    qnxt.assign(q, -1);

    // ---------- 逐组处理 ----------
    for (int g = 0; g < (int)grules.size(); g++) {
        n1 = n2 = 0;
        newNode1(); newNode2();    // 根节点编号都是 0
        // 建 Trie1：插入每条规则的 reverse(pre)
        for (int t = 0; t < (int)grules[g].size(); t++) {
            int id = grules[g][t], u = 0;
            for (int p = 0; p < (int)rpre[id].size(); p++) {
                int x = rpre[id][p] - 'a';
                if (!ch1[u][x]) ch1[u][x] = newNode1();
                u = ch1[u][x];
            }
            rnxt[id] = rhead[u]; rhead[u] = id;  // 规则挂到结尾节点
        }
        // 询问挂到 Trie1 节点：reverse(TA) 走到的最深节点
        for (int t = 0; t < (int)gqs[g].size(); t++) {
            int id = gqs[g][t], u = 0;
            for (int p = 0; p < (int)qU[id].size(); p++) {  // 走断就停，更短的 pre 仍可能匹配
                int x = qU[id][p] - 'a';
                if (!ch1[u][x]) break;
                u = ch1[u][x];
            }
            qnxt[id] = qhead[u]; qhead[u] = id;
        }
        // DFS Trie1（用显式栈，防止递归太深爆栈）
        // 全程维护不变量：Trie2 里装着"当前路径上挂着的所有规则的 suf"
        vector<int> st_u, st_i;
        st_u.push_back(0); st_i.push_back(0);
        enter(0);
        while (!st_u.empty()) {
            int u = st_u.back(), i = st_i.back();
            if (i < 26) {
                st_i.back()++;
                int v = ch1[u][i];
                if (v) { st_u.push_back(v); st_i.push_back(0); enter(v); }
```

```
            } else {
                quit(u);
                st_u.pop_back(); st_i.pop_back();
            }
        }
    }
    for (int i = 0; i < q; i++) printf("%lld\n", ans[i]);
    return 0;
}
```

样例验证：第一个询问 t1 = xabcx，t2 = xadex，差异段是 bc → de，TA = xa，TC = x。同组规则有两条：规则 1（pre = xa，suf = x）和规则 3（pre = suf = 空）。把 U = reverse(TA) = ax 放到 Trie1 上走，路径是 根 → a → x：规则 3 的 reverse(pre) 为空、挂在根上，规则 1 的 reverse(pre) = ax、恰好挂在路径末端的节点 x 上——两条都是候选。DFS 到节点 x 时，Trie2 里同时装着这两条规则的 suf（空串在根计数，x 在节点 x 计数），把 TC = x 放上去走，沿途计数之和 = 1（空串）+ 1（x）= 2，与样例一致。

T4 员工招聘（employ）—— 计数 DP + 容斥

题目原文

【题目描述】小 Z 和小 H 想要合伙开一家公司，共有 n 人前来应聘，编号为 1 ~ n。小 Z 和小 H 希望录用至少 m 人。

小 H 是面试官，将在接下来 n 天每天面试一个人。小 Z 负责决定应聘人前来面试的顺序：选择一个 1 ~ n 的排列 p，在第 i 天通知编号为 p_i 的人前来面试。

第 i 天的面试题难度为 $s_i \in \{0, 1\}$：$s_i = 0$ 表示没有人能做出，$s_i = 1$ 表示所有人都能做出。小 H 根据作答结果决定是否录用：没做出则拒绝，否则录用。

然而，编号为 i 的人有一个耐心上限 c_i：若在他之前已经有不少于 c_i 人被拒绝或放弃参加面试，则他也将放弃参加面试。

小 Z 想知道一共有多少种面试顺序 p 能够让他们录用至少 m 人。答案对 998244353 取模。

【输入格式】从文件 employ.in 中读入数据。第一行包含 n, m；第二行包含长度为 n 的字符串 $s_1 \cdots s_n$；第三行包含 n 个非负整数 $c_1, \ldots, c_n$。

【输出格式】输出到文件 employ.out 中，一行一个非负整数。

【样例 1 输入】

```
3 2
101
1 1 2
```

【样例 1 输出】

```
2
```

【样例 1 解释】共有 2 种顺序：p = [1, 2, 3]（依次录用 1 号和 3 号）；p = [2, 1, 3]（依次录用 2 号和 3 号）。

一句话概括：安排 n 个人的面试顺序，第 i 天 $s_i = 1$ 才能录用；一个人若在他之前"被拒 + 放弃"的人数已达到 c_i 就会放弃。问有多少种排列能让录用人数 $\geqslant m$，对 998244353 取模。

数据范围

对于所有数据：$1 \leqslant m \leqslant n \leqslant 500$；$s_i \in \{0, 1\}$；$0 \leqslant c_i \leqslant n$。共 25 个测试点，每个 4 分。时限 1.0 秒，内存 512 MiB。

测试点	n ⩽	m	特殊性质

1、2	10	$\leqslant n$	无
3~5	18	$\leqslant n$	无
6~8	100	$\leqslant n$	A
9 ~ 11	100	$\leqslant n$	无
12 ~ 14	500	$= 1$	无
15	500	$= n$	无
16、17	500	$\leqslant n$	A
18 ~ 21	500	$\leqslant n$	B
22 ~ 25	500	$\leqslant n$	无

特殊性质 A：所有 $s_i = 1$；特殊性质 B：s 中最多只有 18 个 1。

第 1 档：暴力全排列（8 分）

$n \leqslant 10$ 时（测试点 1、2），`next_permutation` 枚举全部 n! 个排列，每个排列 O(n) 模拟一遍过程即可。$10! \times 10 \approx 3.6 \times 10^7$，能过。

```
#include <bits/stdc++.h>
using namespace std;
int n, m, c[15];
char s[15];
int main() {
    freopen("employ.in", "r", stdin);
    freopen("employ.out", "w", stdout);
    scanf("%d%d%s", &n, &m, s + 1);
    for (int i = 1; i <= n; i++) scanf("%d", &c[i]);
    vector<int> p(n);
    for (int i = 0; i < n; i++) p[i] = i + 1;
    long long ans = 0;
    do {
        int bad = 0, hire = 0;        // bad = 被拒+放弃人数
        for (int i = 1; i <= n; i++) {
            int x = p[i - 1];
            if (bad >= c[x]) bad++;        // 放弃
            else if (s[i] == '1') hire++;  // 录用
            else bad++;                    // 被拒
        }
        if (hire >= m) ans++;
    } while (next_permutation(p.begin(), p.end()));
    printf("%lld\n", ans);
    return 0;
}
```

$n \leqslant 18$ 的测试点（3 ~ 5）可以用状态压缩 DP（$O(2^n \cdot n)$）再拿 12 分，思路是把"已安排的人的集合"压进状态。这里不展开（本档代码从略），因为我们的目标是满分。

第 2 档：把过程抽象出来

仔细看这个过程，有两个至关重要的观察：

观察 1：不管怎么排列，第 i 天结束后，"被拒 + 放弃"的人数 x 与录用人数 h 永远满足 $\mathbf{x + h = i}$（每天必有一人面试，结果非此即彼）。也就是说，只要知道 x，录用人数 $h = i - x$ 自动确定。

观察 2：$s_i = 0$ 的日子是"废日"——无论谁来，结果都是 x 加 1（做不出被拒，或者干脆放弃），对选谁没有任何限制。真正需要精心挑选人的，只有 $s_i = 1$ 的日子。在 $s_i = 1$ 的日子安排一个人，设当前 $x = j$：

- 安排一个 $c > j$ 的人 → **录用**（x 不变）；
- 安排一个 $c \leqslant j$ 的人 → **放弃**（x 加 1）。

难点：x 在变，"$c \leqslant x$ 的人"这个集合在不断扩大，而且我们必须知道还剩多少个 $c \leqslant x$ 的人没被用过才能计数。直接记录"哪些人用过"状态爆炸，这就是本题最难的地方。

第 3 档：满分 —— 容斥 + "钦定"思想

突破口是一个经典的计数技巧——**容斥**。在 $s_i = 1$ 的日子，"安排 $c > j$ 的人录用"这个条件不好处理，我们把它拆成两部分：

$$\text{安排 } c > j \text{ 的人} = \text{随便安排一个人} - \text{安排一个 } c \leqslant j \text{ 的人}。$$

也就是说，我们允许自己"违规"安排 $c \leqslant j$ 的人去录用，只要给这种方案乘上 −1 的容斥系数，在总和里它就会把多算的部分减掉。

DP 状态设计：设 $f(j, k)$ 表示按顺序处理到某个位置时，当前 $x = j$（被拒 + 放弃共 j 人），且已经"钦定填了具体人"的位置有 k 个的方案数（含容斥正负号）。

预处理 d_j = 满足 $c \leqslant j$ 的人数（排个序或桶计数即可）。

每到一个 $s_i = 1$ 的日子（设当前 $x = j$），有三种转移：

1. **录用，不钦定具体人**：方案数不变，$f(j, k) \to f(j, k)$。（"随便安排"的那部分，人选留到最后统一用排列数补齐。）
2. **录用，但钦定是违规的（$c \leqslant j$）**：要从 $c \leqslant j$ 的人里挑一个还没钦定过的，有 $(d_j - k)$ 种选法，容斥系数 −1：$f(j, k) \to -(d_j - k) \cdot f(j, k)$，同时 $k + 1$。
3. **放弃（不录用）**：同样从 $c \leqslant j$ 的人里挑一个，$(d_j - k)$ 种选法：$f(j, k) \to (d_j - k) \cdot f(j, k)$，同时 $j + 1$、$k + 1$。

每到一个 $s_i = 0$ 的日子：必然 $x + 1$，且不钦定人，$f(j, k) \to f(j + 1, k)$。

为什么系数是 $(d_j - k)$? $c \leqslant j$ 的人共 d_j 个，之前每一次"钦定"（不管是违规录用还是放弃）都用掉了其中一个，所以还剩 $d_j - k$ 个可选。而那些"不钦定"的位置不占这个池子——这正是容斥的代数结构保证的（标准的"限制位置计数"技巧，记住这个套路即可）。

统计答案：DP 结束时，被钦定的 k 个位置已经填了具体的人；其余 $n - k$ 个位置（包括所有 $s_i = 0$ 的日子和"不钦定"的录用位）可以随便安排剩下的人，乘 $(n-k)!$ 即可。最终：

$$\text{ans} = \sum_{j=0}^{n-m} \sum_{k} f(j,k) \times (n-k)!$$

（$j \leqslant n-m$ 等价于录用人数 $n-j \geqslant m$。注意 f 里已经带好了容斥的正负号，直接相加。）

复杂度：状态 $O(n^2)$，转移 $O(1)$。三重循环 i, j, k 的总迭代次数约为 $n^3/3 \approx 4 \times 10^7$（$n = 500$），1 秒内轻松跑完。空间用滚动数组 $O(n^2)$。

```
#include <bits/stdc++.h>
using namespace std;
const int mod = 998244353;
int n, m;
char s[505];
int d[505];                 // d[j] = 满足 c <= j 的人数
long long fac[505];
long long f[505][505];      // f[j][k]：x = j，钦定了 k 个位置
long long g[505][505];
int main() {
    freopen("employ.in", "r", stdin);
    freopen("employ.out", "w", stdout);
    scanf("%d%d%s", &n, &m, s + 1);
    for (int i = 1; i <= n; i++) {
        int c; scanf("%d", &c);
        d[c]++;
    }
    for (int i = 1; i <= n; i++) d[i] += d[i - 1];
    fac[0] = 1;
    for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % mod;
    f[0][0] = 1;
    for (int i = 1; i <= n; i++) {
        memset(g, 0, sizeof(g));
        for (int j = 0; j < i; j++) {
            for (int k = 0; k <= i; k++) {
                // 注意：钦定录用会让 k > j，上界要开到 i
                long long v = f[j][k];
                if (!v) continue;
                long long pool = d[j] - k;   // c <= j 且未被钦定的人数
                if (s[i] == '0') {
                    // 废日：x + 1，不钦定
                    g[j + 1][k] = (g[j + 1][k] + v) % mod;
                } else {
                    // 录用，不钦定
                    g[j][k] = (g[j][k] + v) % mod;
                    // 录用，钦定违规（容斥 -1）
                    g[j][k + 1] = (g[j][k + 1] - v * pool % mod + mod) % mod;
                    // 放弃：x + 1
                    g[j + 1][k + 1] = (g[j + 1][k + 1] + v * pool % mod) % mod;
```

```
                }
            }
        }
        memcpy(f, g, sizeof(f));
    }
    long long ans = 0;
    for (int j = 0; j <= n - m; j++)
        for (int k = 0; k <= n; k++)
            ans = (ans + f[j][k] * fac[n - k]) % mod;
    printf("%lld\n", (ans % mod + mod) % mod);
    return 0;
}
```

样例验证：$n = 3$，$m = 2$，$s = 101$，$c = \{1, 1, 2\}$。DP 结束后 $f(1, 0) = 1$，$f(1, 1) = -2$，答案 $= 3! \times 1 + 2! \times (-2) = 6 - 4 = 2$，与样例输出一致。

结语：考场得分策略小结

题目	核心考点	保底部分分	满分关键
T1 club	贪心	暴力 3^n / 特殊性质 A、B	先选最爱 + 按损失反悔
T2 road	MST、枚举	k = 0 纯 Kruskal	枚举乡镇子集 + 原图浓缩成 MST
T3 replace	字符串	暴力比对	差异段分组 + 双字典树
T4 employ	计数 DP	全排列暴力	容斥 + 钦定 DP

最后送大家三句话：

1. **部分分是真金白银**。哪怕只想得出暴力，也一定要写下来——暴力往往就是 20 ~ 40 分，四题凑一凑就过二级线了。
2. **特殊性质是出题人的提示**。性质 A、B 往往就是满分做法的简化版，先想明白特殊情况，一般情况常常水到渠成。
3. **数据结构是乘法器**。T2 的并查集、T3 的哈希与字典树、T4 的阶乘与桶计数，都是提高组的基本功，平时练熟，考场才能信手拈来。

祝各位同学认证顺利，一战上岸！